LT10d : ADT Hash Table

Part 3 : Collision Resolutions

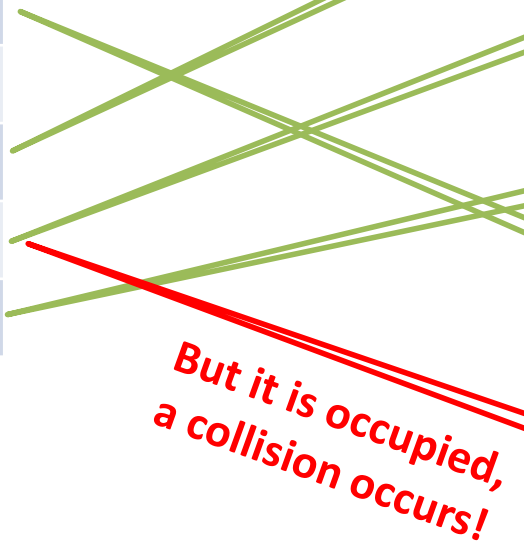
What is a Collision?

A collision occurs when two or more elements are hashed to a same value.

```
>>> data = ['cdab','dbac','dabc','bdac','badc']
```

Array Index	
0	'bdac'
1	
2	'cdab'
3	'dbac'
4	'dabc'

Hash value % 5	Hash value	string
2	982	'cdab'
3	983	'dbac'
4	984	'dabc'
0	985	'bdac'
3	988	'badc'



Collision Resolution #1: Open Hashing

"Separate Chain" : create a separate list at the particular index position.

```
>>> data = ['cdab','dbac','dabc','bdac','badc']
```

Array Index	
0	'bdac'
1	
2	'cdab'
3	['dbac', 'badc']
4	'dabc'

For every element in a given list, find the hash value, **i**, of the element.

- fill it in the hash table at index **i** if it is empty;
- if it is not empty and ***there is a list***, add the element into the list;
- if it is not empty and ***there is no list***, replace it with a list containing both the elements.

Hash Table (with "Separate Chain"):

```
def hashtable(seq):
    tbl = init_table(len(seq))
    for ele in seq:
        i = hash(ele) % len(seq)
        if tbl[i] == '':
            tbl[i] = ele
        else: #Separate Chain


    return tbl
```

Write the Python code for Collision Resolution #1: 'Separate Chain'

Searching an item in a Hash Table (with "Separate Chain"):

Array Index	
0	'bdac'
1	
2	'cdab'
3	['dbac', 'badc']
4	'dabc'

Hint : The operation **`item in table[i]`** is a linear search operation.

Find the hash value, **i**, of the item your want to search.

- it *does not exist* if the hash table at index **i** is empty;
- it *exists* if the hash table at index **i** is not empty and is what you want to find;
- it *does not exist* if the hash table at index **i** is not empty and is not what you want to find;
- if there is a list in the hash table at index **i**, perform a **linear search** on the list which will return a True or False.

Searching an item in a Hash Table (with "Separate Chain"):

```
def search(table, item):
```

Write the Python code for searching an item
in a Hash Table with 'Separate Chain'.

```
>>> data = ['cdab','dbac','dabc','bdac','badc']

>>> data_table = hashtable(data)

>>> search(data_table, 'cdab')
>>> search(data_table, 'adcb')
>>> search(data_table, 'dbac')
>>> search(data_table, 'bcda')
```

data_table

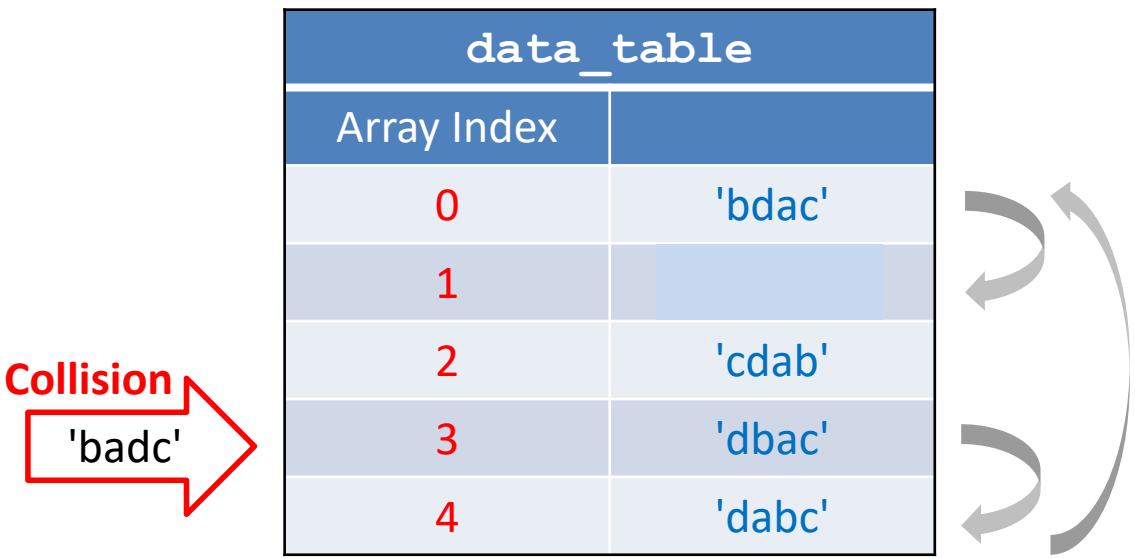
Array Index	
0	'bdac'
1	
2	'cdab'
3	['dbac', 'badc']
4	'dabc'

Collision Resolution #2: Closed Hashing

"Linear Probing" - Starting from it's "assigned location", look for the next nearest empty position. When it reaches the end of the array, it will continue searching from the front until an available location is found.

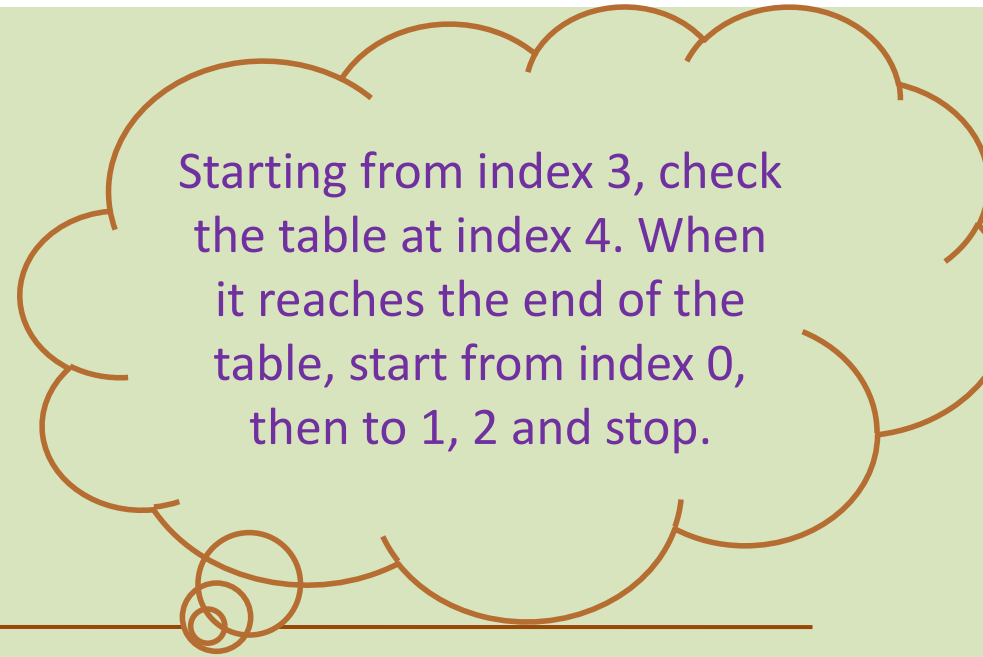
```
>>> data = ['cdab','dbac','dabc','bdac','badc']

>>> data_table = hashtable(data)
```



Starting from index 3, check the table at index 4. When it reaches the end of the table, start from index 0, then to 1, 2 and stop.

Core – *Skill* :



Iterate through every element in a list once starting from an index position

Core Skill: Iterate through every element in a list once starting from an index position

```
def browse(seq, pos):
```

Write the Python code for this Core Skill

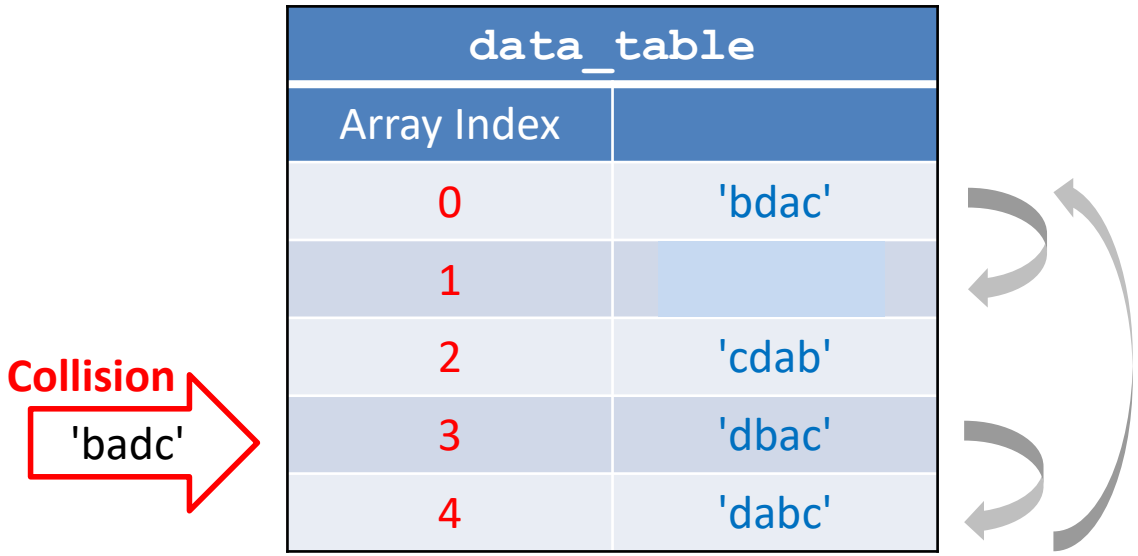
```
    return None
```

```
>>> lst = ['a', 'e', 'i', 'o', 'u']

>>> browse(lst, 2)
>>>   'i'
      'o'
      'u'
      'a'
      'e'
```

Collision Resolution #2: Closed Hashing

"Linear Probing" - Look for the next nearest empty position to fill in the data.



For every element in a given list, find the hash value, **i**, of the element.

- fill it in the hash table at index **i** if it is empty;
- if it is not empty, iterate through the table to find the next nearest empty position. When it reaches the end of the table, start finding from the beginning of the table.

Will there be a situation that no empty position is found and it results in an infinite loop?

Hash Table (with Closed Hashing):

```
def hashtable(seq):
    tbl = init_table(len(seq))
    for ele in seq:
        i = hash(ele) % len(seq)
        if tbl[i] == '':
            tbl[i] = ele
        else:
            #Closed Hashing
    return tbl
```

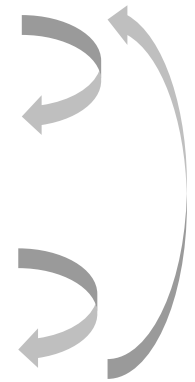
Write the Python code for Collision Resolution #2: Linear Probing

```
>>> data =
['cdab','dbac','dabc','bdac','badc']
```

```
>>> data_table = hashtable(data)
```

Collision
'badc'

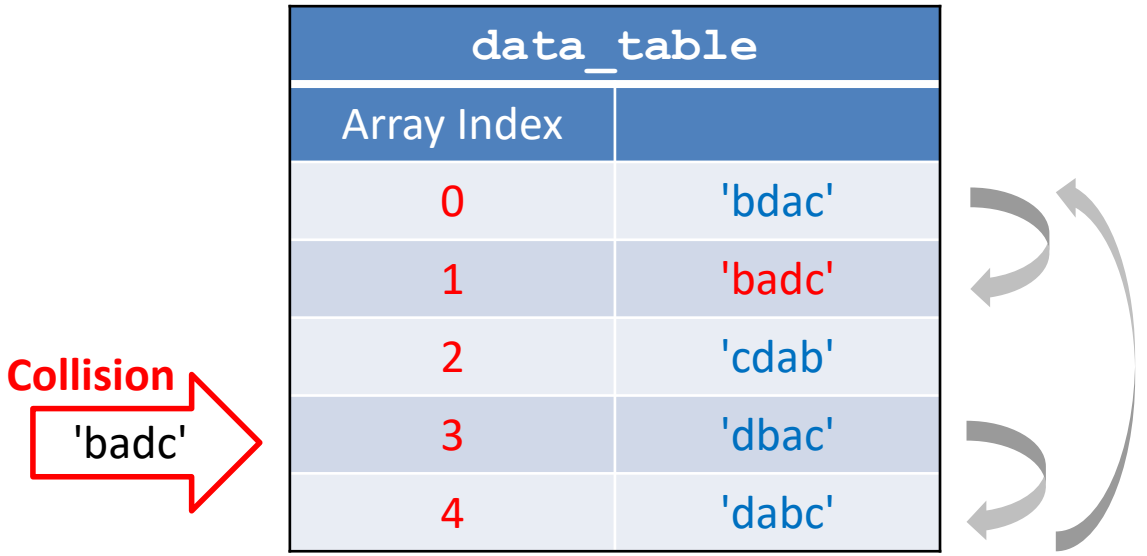
data_table	
Array Index	
0	'bdac'
1	'badc'
2	'cdab'
3	'dbac'
4	'dabc'



Searching a Hash Table (with Closed Hashing):

```
def search(table, item):
    i = hash(item) % len(table)
    if table[i] == '':
        return False
    elif table[i] == item:
        return True
    else:
```

Write the Python code for searching an item in a Hash Table with Linear Probing.




```
>>> search(data_table, 'cdab')
>>> search(data_table, 'adcb')
>>> search(data_table, 'dbac')
>>> search(data_table, 'bcda')
```

The End